



Nombre y Apellidos:
Sección:

RUT:
Firma:

CC1002 – 2024 Semestre Primavera - Control 2 - Duración: 2:00 horas

Indicaciones Generales:

- 1.- Lea atentamente las preguntas. **Habrá dos rondas de preguntas de enunciado durante el control.**
- 2.- Debe responder en las hojas de respuestas que se le entregarán, si requiere más, debe solicitarlas – no se aceptarán otros tipos de hojas.
- 3.- Siempre debe incluir la receta de diseño al escribir una función (contrato, propósito, ejemplos y al menos una prueba).
- 4.- Puede importar y usar los módulos vistos en clases para definir y procesar listas y árboles.
- 5.- Recuerde que `listaVacia` representa al valor `None`.

Pregunta 1 (50%)

Las siguientes instrucciones permiten definir variables y listas con datos de personas:

```
# Persona: identificador(int) nombre(str)
estructura.crear('Persona', 'identificador nombre')

P1 = Persona(101, 'Ana Rios')      #ejemplo de variable de tipo persona
P2 = Persona(213, 'Juan Baeza')   #ejemplo 2
P3 = Persona(21, 'María Caro')    #ejemplo 3
LP = lista(P1, lista(P2, lista(P3, listaVacia))) #lista de personas
```

Con esto, realice lo siguiente:

(A) (2.0 ptos.) Escriba la función `buscarPersona(Lpersonas, id)`, que recibe una lista de Personas y un identificador, y entrega la Persona que tiene ese identificador. Suponga que no hay personas repetidas en la lista, y que todas las personas tienen un identificador distinto. Si no existe una persona con el identificador indicado dentro de la lista, entonces la función entrega `None`. Ejemplos:

- `buscarPersona(LP, 213)` entrega: `Persona(213, 'Juan Baeza')`
- `buscarPersona(LP, 33)` entrega: `None`

(B) (4.0 ptos.) Escriba la función `listaPersonas(Lpersonas, Lids)`, que recibe una lista de personas y una lista de identificadores, y entrega una lista con las personas que tengan tales identificadores (sin incluir las que no existan en la lista de personas). Ejemplo:

- Si tenemos la lista de identificadores
`Li = lista(21, lista(33, lista(101, listaVacia)))`
entonces `listaPersonas(LP, Li)` entrega: `lista(P3, lista(P1, listaVacia))`

Solución Pregunta 1:

P1.A [2.0 ptos]

```
# receta de diseño: 0.3
# buscarPersona: lista(Persona) int -> Persona #0.1
# Devuelve la persona en Lpersonas con el identificador id #0.1
# Ej.: buscarPersona(L, 213) entrega P2 y buscarPersona(L, 0) entrega None #0.1
def buscarPersona(Lpersonas, id):
    # Caso lista vacia: 0.3
    if vacia(Lpersonas): #0.2
        return None #0.1
    else:
        #caso identificador en cabeza de lista: 0.8
        persona = cabeza(Lpersonas) #0.1
        if persona.identificador == id: #0.5
            return persona #0.2
```



Nombre y Apellidos:
Sección:

RUT:
Firma:

```
    else:
        #caso recursivo: 0.4
        return buscarPersona(cola(Lpersonas), id) #0.4

# Test #0.2
assert buscarPersona(L, 213) == P2
assert buscarPersona(L, 0) == None

# Pl.B [4.0 ptos]

# Primera solucion:

# receta de diseño: 0.3
# listaPersonas: lista(Persona) lista(int)-> lista(Persona) #0.1
# lista con personas de lista L con identificadores en lista de numeros #0.1
# ej: listaPersonas(lista(21,lista(0,lista(101,None))),L)-> #0.1
#     lista(P3,lista(P1,None))

def listaPersonas(Lpersonas, Lids):
    #caso base: 0.5
    if vacia(Lids): #0.3
        return listaVacía #0.2

    #caso general: 3.0
    P = buscarPersona(Lpersonas, cabeza(Lids)) #0.5
    if P == None: #0.5
        return listaPersonas(Lpersonas, cola(Lids)) #0.8
    else: #0.2
        return lista(P, listaPersonas(Lpersonas, cola(Lids))) #1.0

# Test #0.2
assert listaPersonas(L, lista(21, lista(0, lista(101, None)))) == lista(P3, lista(P1, None))

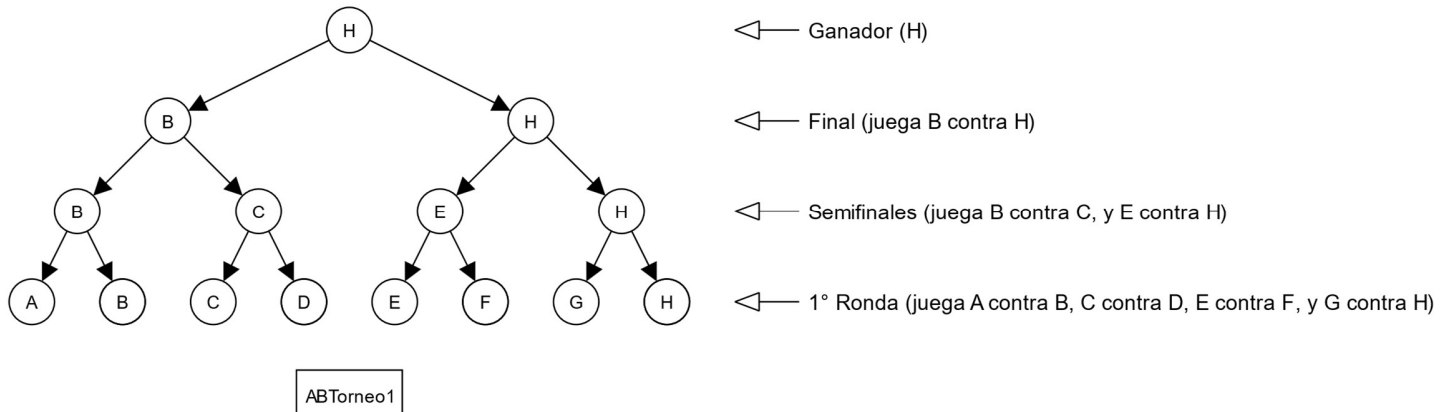
# Solucion alternativa: con funciones abstractas

# receta de diseño: 0.3
# listaPersonas: lista(Persona) lista(int) -> lista(Persona) #0.1
# Devuelve la lista de las personas en Lpersonas que tengan los #0.1
# identificadores en Lids (sin incluir las que no estén en Lpersonas)
# Ej: listaPersonas(lista(21, lista(0, lista(101, listaVacía))), LP) devuelve #0.1
# lista(P3, lista(P1, listaVacía))
def listaPersonas(Lpersonas, Lids):
    resultado = mapa(lambda x: buscarPersona(Lpersonas, x), Lids) #1.75
    return filtro(lambda x: x != None, resultado) #1.75
    # Tambien se puede hacer el filtro primero y luego el mapa
    # resultado = filtro(lambda x: buscarPersona(Lpersonas, x) != None, Lids)
    # return mapa(lambda x: buscarPersona(Lpersonas, x), resultado)

# Test #0.2
assert listaPersonas(L, lista(21, lista(0, lista(101, listaVacía)))) ==
    lista(P3, lista(P1, listaVacía))
```

Pregunta 2 (50%)

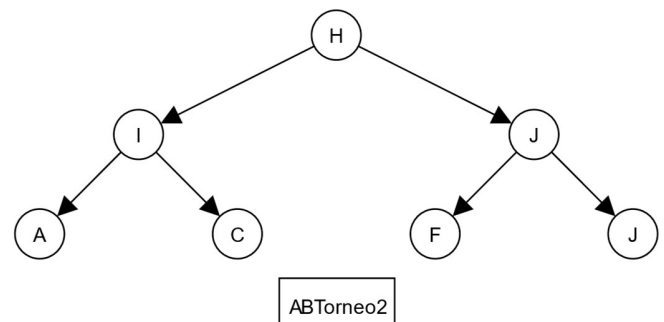
Los resultados de un campeonato deportivo por eliminación se representan en un árbol binario (AB) que contiene Strings. Por ejemplo, el árbol binario **ABTorneo1** de la figura representa un campeonato donde el ganador fue **H** (note que **H** jugó contra **G**, **E** y **B**):



El número de rondas del campeonato de un árbol es desconocido. Suponga que todas las rondas están llenas (es decir, se jugaron todos los juegos de todas las rondas).

Note que, en un árbol correcto (como el de arriba), el valor de un nodo siempre corresponde al valor de alguno de sus hijos (el que ganó el juego).

Por otro lado, el árbol binario **ABTorneo2** de la figura de la derecha, no representa un árbol de torneo válido, pues no se cumple lo anterior (ninguno de los hijos del nodo H tiene su mismo valor).



Considerando la siguiente definición de árbol binario:

```
# AB: valor(str) izq(AB) der(AB)
estructura.crear('AB', 'valor izq der')
```

Y suponiendo que existen las variables **ABTorneo1** y **ABTorneo2**, que definen los árboles de ejemplo de arriba, escriba las siguientes funciones:

(A) Escriba la función **validar(A)**, que recibe un árbol binario, y entrega **True** si se encuentra construido correctamente, representando un torneo válido (y **False** en cualquier otro caso). Ejemplos:

- **validar(ABTorneo1)** entrega: **True**
- **validar(ABTorneo2)** entrega: **False**

(B) Escriba la función **rivales(A)**, que recibe un árbol binario que representa un torneo, y entrega una lista con todos los rivales que fueron derrotados por el ganador del torneo. Ejemplo:

- **rivales(ABTorneo1)** entrega: **lista('B', lista('E', lista('G', listaVacía)))**

Nota: El orden de los rivales en la lista no es importante. Puede entregarlos en el orden que estime conveniente dentro de la lista.



Nombre y Apellidos:
Sección:

RUT:
Firma:

Solución Pregunta 2:

```
# P2.A [3 ptos]

# receta de diseño: 0.3
# validar: AB -> bool #0.1
# devuelve True si el árbol binario corresponde a un torneo, False sino #0.1
# ejemplo: validar(ABTorneo1) devuelve True #0.1
def validar(arbol):
    if arbol == None: #0.3
        return False #0.2
    elif arbol.izq == None and arbol.der == None: #0.6
        return True #0.2
    else: #1.2
        return (validar(arbol.izq) and validar(arbol.der)) and
            (arbol.valor == arbol.izq.valor or arbol.valor == arbol.der.valor)
            and (arbol.izq.valor != arbol.der.valor)

# Test #0.2
assert validar(ABTorneo1)
assert not validar(ABTorneo2)

# P2.B [3 ptos]

# receta de diseño: 0.3
# rivales: AB -> lista(str) #0.1
# devuelve una lista de los rivales derrotados por el ganador del torneo #0.1
# ejemplo: rivales(ABTorneo1) devuelve lista('B', lista('E', lista('G', listaVacía))) #0.1
def rivales(arbol):
    if arbol.izq == None and arbol.der == None: #0.4
        return listaVacía #0.3
    else:
        if arbol.valor == arbol.izq.valor: #0.4
            return lista(arbol.der.valor, rivales(arbol.izq)) #0.7
        else:
            return lista(arbol.izq.valor, rivales(arbol.der)) #0.7

# Test #0.2
assert rivales(ABTorneo1) == lista('B', lista('E', lista('G', listaVacía)))
```